

AgentCompile: An LLM-Guided Compiler for Direct CUDA Inference

Xuanzhe Li^{1,2}, Ziyang Weng¹, Zhiyu Zhu^{1,2*}, Junhui Hou²

¹City University of Hong Kong (Dongguan)

²City University of Hong Kong

lixuanzhede29@126.com, 72510283@cityu-dg.edu.cn, zhiyu.zhu@cityu-edu.dg.cn, jh.hou@cityu.edu.hk

Abstract

Transformer inference increasingly relies on specialized compiler and runtime support, while recent LLMs can generate nontrivial CUDA kernels. However, unconstrained generation guarantees neither correctness nor performance. We present AGENTCOMPILE, an LLM-guided CUDA inference compiler that combines two complementary uses of LLMs. First, the LLM provides advisory metadata for compiler-derived region summaries and bounded candidate spaces. The compiler then instantiates template-based CUDA candidates, validates correctness, selects implementations by measured latency, and falls back when specialization is unsupported or unprofitable. Second, under compiler-defined contracts, the LLM directly generates five classes of decode-critical kernels to accelerate inference, prompted by distilled optimization principles. AGENTCOMPILE integrates these kernels into a serving runtime with paged KV cache, continuous batching, preemption, chunked prefill, and bucketed full-step CUDA Graph replay. Across six evaluated model families, AGENTCOMPILE achieves speedups of **2.23–6.98×** over PyTorch eager for single-request generation, and **1.04–1.16×** over vLLM for both single-request generation and multi-request serving. Our code is publicly available at <https://github.com/veneno1213822/AgentCompile>.

Introduction

Modern transformer inference relies on compilers and runtimes such as PyTorch 2, FlashAttention, and vLLM (Ansel et al. 2024; Dao et al. 2022; Kwon et al. 2023), which map model programs to efficient accelerator execution. Yet each path leaves a gap during autoregressive decoding: PyTorch eager retains substantial Python dispatch and dynamic-cache overhead; `torch.compile` suffers graph breaks, dynamic cache updates, and shape variation under long-context generation; serving runtimes such as vLLM rely on a fixed library of expert-written kernels. As our experiments show, these overheads compound to an end-to-end latency gap of up to $\sim 7\times$ for single-request generation. Directly asking an LLM to write the missing CUDA kernels is risky: correctness hinges on low-level details from indexing and synchroniza-

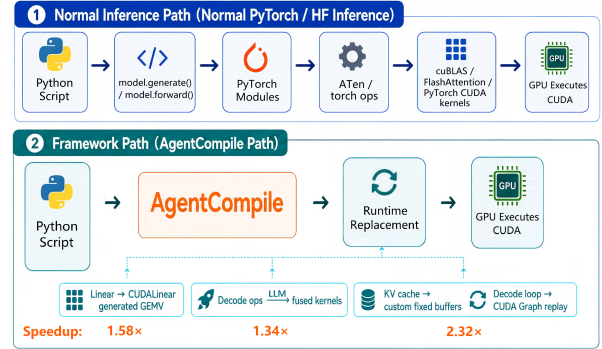


Figure 1: Comparison of the standard PyTorch inference stack and AGENTCOMPILE.

tion to launch configuration, and even a correct kernel is useless if it is slower than the library kernel it replaces.

We find, however, that these risks can be contained. When generation is driven by distilled optimization principles rather than reference source code, and every candidate must pass compiler-side gates, an explicit performance-acceptance threshold, and in-graph measured selection before deployment, LLM-authored kernels can match or exceed expert-written and library kernels on real decode workloads, while a fallback path guarantees that the system never regresses when they do not.

We present AGENTCOMPILE, a compiler framework that combines two uses of LLMs. In the compiler pipeline, the LLM only advises: it labels compiler-constructed regions and prioritizes bounded template candidates, while the compiler generates and checks all code. On top of the pipeline, the LLM directly authors complete CUDA kernels for five decode-critical fusion patterns that span multiple compiler regions, which is beyond what template-based generation can express, and the compiler’s role shrinks to checking: validation, measurement, and admission. This replaces what were previously hand-written, model-specific fused kernels and attention bodies, which rarely transferred across architectures, with kernels re-derived per model from the same principle libraries, so that supporting a new model requires generation rather than re-engineering. Both tiers feed a serving-grade runtime with paged KV cache, continuous batching,

*Corresponding author.

and full-step CUDA Graph replay. Figure 1 positions AGENT-COMPILE as a substitution layer between Python model code and CUDA execution.

This paper makes three contributions: (i) a compiler-controlled pipeline that maps LLM advice to template-generated CUDA candidates; (ii) a principle-driven LLM kernel-generation methodology; and (iii) a serving-grade hybrid runtime evaluated across six model families including a vision-language model, outperforming PyTorch eager and `torch.compile` on single-request generation, and vLLM on both single-request generation and multi-request serving.

Related Work

Neural-network compilers: Compiler stacks and auto-schedulers lower models to accelerators through templates, schedules, and rewrites (Chen et al. 2018a; OpenXLA Project 2024; Ansel et al. 2024; Lattner et al. 2021; Tillet, Kung, and Cox 2019; Chen et al. 2018b; Zheng et al. 2020; Zhu et al. 2022; Shi et al. 2023; Jia et al. 2019; Niu et al. 2021; Zheng et al. 2022; Ma et al. 2020); AGENT-COMPILE is complementary, adding an LLM-authored kernel tier on top of compiler-defined constraints, validation, and measurement. **Inference runtimes and kernels:** Serving runtimes and specialized kernels (Kwon et al. 2023; Yu et al. 2022; NVIDIA 2024; Dao et al. 2022) optimize scheduling and decode execution; AGENT-COMPILE implements the same serving-grade mechanisms but sources its decode kernels from contract-gated LLM generation rather than a fixed expert-written library. **LLMs for code and compiler optimization:** Unlike direct LLM kernel generation as evaluated by KernelBench (Ouyang et al. 2025) and related LLM-for-code work (Chen et al. 2021; Li et al. 2022, 2023; Shinn et al. 2023; Ni et al. 2024; Cummins et al. 2023), AGENT-COMPILE renders its prompts from distilled principle libraries (including falsified dead ends) rather than task descriptions or reference source, and every kernel must pass compiler contracts, acceptance thresholds, and runtime measured admission with a never-worse fallback. An extended discussion of related work is provided in the technical appendix.

Problem Formulation and Design Goals

Existing CUDA-based inference pipelines are built from fixed, hand-engineered kernels and hard-wired execution paths, so they offer little customization for a given model and cannot adapt their kernels to the structure of the network being served. Handing the task to an LLM does not by itself resolve this: freely generated CUDA code offers no guarantee of correctness or speed, and an unverified kernel cannot be trusted inside a production decode loop. The problem is therefore to admit LLM output, whether advisory metadata or complete kernel source, into an inference compiler such that it becomes executable only after passing compiler-controlled contracts, empirical validation, and measured selection.

We model an inference program as a directed graph $G = (V, E)$ of tensor operations and data dependencies. A region $r \subseteq V$ is a connected subgraph with an external interface (tensors, dependencies, dtype/layout assumptions, reduction axes, calling convention); a CUDA replacement is admissible

only if it preserves this interface and passes the validation protocol of the surrounding inference path.

We decompose the problem into two tiers of LLM involvement, both governed by a shared admission discipline: Tier 1 uses the LLM only as an advisor over compiler-enumerated template candidates, Tier 2 lets the LLM author complete kernels under compiler-defined contracts, and a runtime admission layer decides by measurement whether any validated kernel actually runs.

Tier 1: advisory search over template candidates. For a region r and hardware configuration H , the compiler constructs an executable candidate space $\mathcal{C}(r, H)$ of template implementations with realizable parameters (full list in the appendix). The LLM observes a compiler-generated region summary $S(r)$ rather than raw source code and returns advisory metadata

$$\mathcal{A}_{\text{LLM}}(S(r), \mathcal{C}(r, H)) = (\ell_r, \pi_r, \eta_r, \rho_r),$$

where ℓ_r is a semantic label, π_r is a candidate preference order, η_r contains parameter hints, and ρ_r contains risk annotations, all of which are treated as hypotheses, with unsupported labels, invalid parameters, and boundary-crossing suggestions rejected by compiler checks.

Tier 2: contract-gated kernel generation. For a recognized fusion pattern p (the five families detailed in Method), the compiler defines a contract $\Sigma(p)$, which specifies the tensor interface, dtype rules, numerical tolerances, and a performance floor, together with a principle library $P(p)$ of distilled optimization knowledge that contains no kernel source. The LLM produces complete kernel source $k = \text{LLM}(P(p), \Sigma(p))$, which enters the validated set $\mathcal{V}(p)$ only if it compiles, satisfies the interface and dtype rules, matches reference outputs within tolerance, and reaches a measured acceptance threshold against the existing implementation; generation runs multiple rounds with structured failure feedback, keeping the best candidate.

Runtime admission and never-worse selection. At runtime, each validated kernel is timed *in-graph* against the existing implementation b and selected only if $T_{\text{graph}}(k) \leq \mu T_{\text{graph}}(b)$, with a small margin μ (we use $\mu=1.04$) absorbing measurement noise; anything slower falls back to the hand-written or library path, and among validated candidates the runtime selects $c^* = \arg \min_{c \in \mathcal{V}} T(c)$, so end-to-end performance is never worse than the baseline path by more than the margin. Validation is empirical rather than formal equivalence: it uses numerical comparison under dtype-appropriate tolerances when a reference exists, and structural and end-to-end consistency checks otherwise.

Method

AGENT-COMPILE implements the formulation above as a three-stage compiler pipeline, plus a serving-grade runtime layer. The pipeline captures a PyTorch or HuggingFace model, forms compiler regions, constructs bounded CUDA candidate spaces, and selects executable template kernels through validation and measurement, where the compiler generates and checks all code and the LLM only advises. The LLM

kernel generator then directly authors the five decode-critical kernel families under compiler-defined contracts, where the compiler only checks to ensure correctness. Figure 2 illustrates the compiler pipeline’s key contract on a representative region r5; Figure 3 illustrates how the five kernel families are generated by the LLM kernel generator.

Pipeline Overview. AGENTCOMPILE organizes the pipeline into three stages: **model capture, analysis & generation,** and **validation & benchmarking.**

Model capture: reads the target model code and obtains a traced computation graph with example inputs, tensor shapes, dtypes, layouts, and dependency metadata (Ansel et al. 2024; Reed et al. 2022). **Analysis & generation:** converts the traced graph into CUDA candidates. The graph and its shape, dtype, layout, dependency, and aliasing metadata are given to the *Graph Analyzer*, which partitions the computation into candidate regions and records each region’s operator sequence, tensor interfaces, reduction axes, dominant operation type, memory-access pattern, and dynamic-shape flags. The *Planner* then enumerates legal fusion, schedule, memory, and parallelization choices for each region. These bounded kernel plans, together with their region interfaces and template families, are passed to the *CUDA Code Generator* to instantiate CUDA candidates. **Validation & benchmarking:** compiles, validates, benchmarks, and packages the generated kernels, following the common compile–measure–select pattern (Chen et al. 2018a; Zheng et al. 2020). Invalid candidates are rejected by compilation, interface, and numerical checks when a reference path is available; the fastest validated implementation is selected; and the system outputs CUDA kernel source files, wrappers, headers, metadata, and build scripts.

Model capture and **validation & benchmarking** are standard, well-studied components that can be built with straightforward graph-capture, validation, benchmarking, and packaging logic. We therefore focus on the **Analysis & generation** stage, which is the part that most distinguishes AGENTCOMPILE.

Graph Analyzer: Region Construction

The analyzer decides which operations may form a candidate region before the LLM is consulted. Traversing the IR (intermediate representation) along use-def dependencies, it merges consecutive pointwise, activation, softmax, norm, and reduction nodes under compatible dtype/layout assumptions, treats layout-only nodes as boundary metadata, keeps matmuls as independent regions (attaching simple epilogues when constraints allow), and makes in-place operations, side effects, unresolved aliasing, unsupported dtypes, and ambiguous dynamic shapes hard boundaries. Each accepted region carries its operator sequence, shapes, dtypes, reduction axes, and access pattern, and rule-based recognition assigns semantic categories; the same layer detects the five fusion patterns handed to the LLM kernel generator, using structural preconditions as hard gates so that models violating a precondition are safely skipped for that pattern. The LLM is invoked only where semantic analysis is useful (mixed

regions, complex GEMM (general matrix–matrix multiplication) epilogues, reduction–pointwise regions), and region boundaries are accepted only after deterministic dependency, alias, and side-effect checks.

CUDA Planner

The planner converts each region into a bounded set of kernel plans rather than asking the LLM to invent implementations: a fusion choice (single-kernel, full, partial, none), a template family (tiled or Tensor-Core GEMM, softmax, reductions, normalization, elementwise, decode-oriented GEMV (general matrix–vector multiplication)), and memory/parallel parameters (vector width, shared-memory policy, grid/block shape, warp mapping); constraint filtering and deterministic ranking then bound the search as in Figure 2. For high-value regions, the LLM may reorder top candidates or suggest parameters, which are accepted only if they already belong to the enumerated plan space, and a diversity filter preserves multiple template families.

CUDA Code Generator

Template-Based Code Generator. The code generator materializes checked plans from deterministic templates, which own all executable details (indexing, boundary checks, dtype conversion, synchronization, launch signatures). Decode-oriented GEMV matters most for autoregressive decoding: weights are stored in an $[N, K]$ layout for contiguous row reads, blocks use 128-bit vectorized loads and warp-shuffle reductions, and a dual-path adaptive- N strategy keeps enough blocks resident for large N .

Principle-Driven LLM Kernel Generator. This generator authors complete CUDA kernels for the five recognized fusion patterns; its design rests on the components described below.

The fusion pass routes recognized patterns to five LLM-generated kernel families that together cover the decode-critical path of a transformer layer (Figure 3): (i) *attention-prelude fusion*: QKV projection + RoPE (rotary position embedding), optionally with per-head Q/K RMSNorm; (ii) *gated-MLP fusion*: gate/up projection + SiLU and down projection + residual add, each in GEMV and thin-GEMM variants dispatched by batch size; (iii) *residual–norm/RoPE fusion*: residual add + RMSNorm, plus a standalone RoPE kernel; (iv) *paged decode attention*: the attention body itself, an online-softmax kernel over paged KV with fused cross-CTA (cooperative thread array) merging; (v) *per-operator decode fusion*: an output-projection (`o_proj`) kernel fusing GEMV + residual + post-RMSNorm, and a language-model-head (`lm_head`) + argmax kernel producing token ids without materializing the logits. Every generated kernel competes against the strongest existing implementation at its site (cuBLAS, FlashAttention, or hand-written Triton) under the measured-admission rule; encoder-only models reuse the same machinery through a parallel encoder-kernel family.

To constrain the generation process, we introduce a principle library, in which the prompts are rendered from a per-pattern library of distilled optimization principles plus a performance mandate. The library contains *no kernel source*:

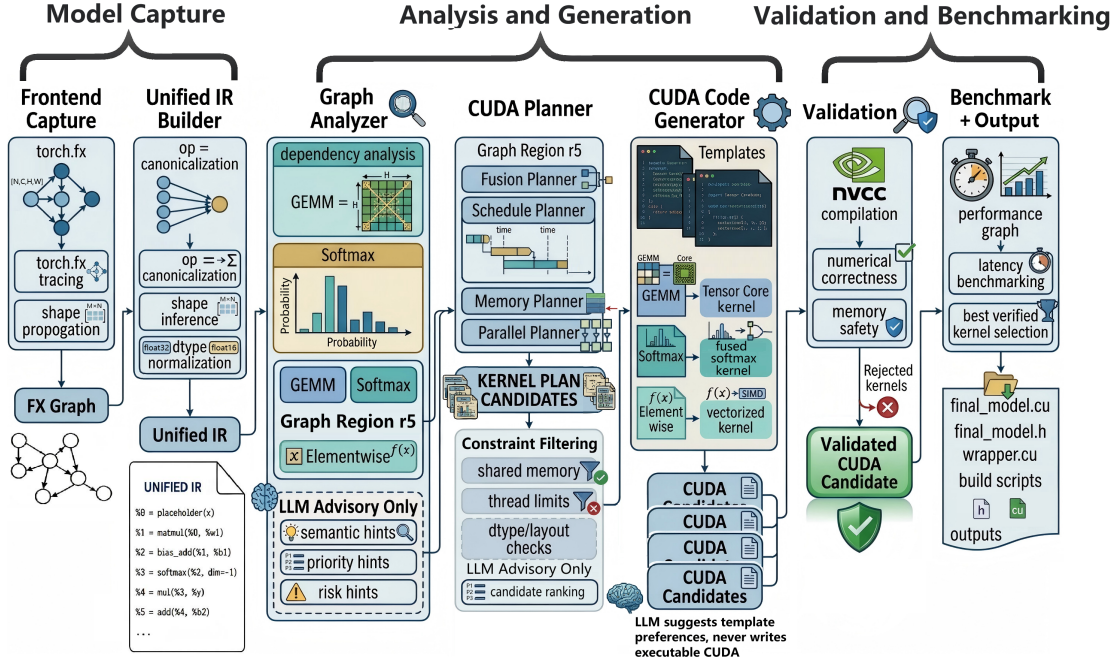


Figure 2: Running example of compiler-guided LLM assistance on a representative region r5. The Graph Analyzer derives a structured region summary for the LLM. The LLM may identify the region as a softmax or normalization pattern, and return semantic labels, candidate preferences, parameter and risk hints (considered hypotheses rather than correctness guarantees); the Planner checks dependencies, reduction semantics, dtype/layout constraints, template availability, and hardware limits before any CUDA is materialized; only plans that pass constraint filtering are sent to the Code Generator and instantiated as CUDA candidates. Once they pass validation checks, the best validated kernels are selected as the final CUDA kernel outputs.

each block records design decisions and, crucially, *falsified dead ends*, that is, approaches that were tried and measured to fail, so that the LLM does not rediscover them. Principles are dimension- and model-agnostic, so the same library serves every captured model and a kernel validated for one model can seed another.

When the principle library cannot cover the pattern at the required level, we hand-write a reference kernel, distill its design decisions and dead ends into prose, and let the LLM re-derive a kernel without ever seeing the reference source. For paged decode attention, the hand-written reference reaches $0.967\text{--}1.00\times$ of FlashAttention at $\sim 90\%$ of A800 peak memory bandwidth (details in the appendix); the LLM-generated kernel reaches $0.966\times$, within 0.1% of the reference.

Each pattern is generated with acceptance threshold $\theta_{\text{gen}}=0.99$ against the existing implementation, at least $N=4$ rounds keeping the best candidate, and informed retry over structured diagnostics (compilation errors, numerical mismatches, slow measurements). Contract clauses are exhaustive about dtypes, since a single unstated dtype assumption can cause every round to fail, and unprofitable sub-families are early-stopped.

Kernel Customization and Multi-Request Adaptation

We further introduce several kernel customizations, including the custom linear layer, KV cache, and attention, to boost

the efficiency of the proposed infrastructure. Meanwhile, to adapt the method to real serving applications, we design serving-level components ranging from globally asynchronous prefill–decode steps to a locally customized paged KV cache mechanism.

CUDALinear replaces PyTorch `nn.Linear` on the decode path, dispatching to custom GEMM/GEMV kernels through a C++ extension with pre-transposed weights and an $M=1$ GEMV path for single-token decoding; prefill and unsupported shapes fall back to cuBLAS. FlashKVCacheManager preallocates fixed-size K/V buffers, tracks per-layer sequence lengths on the GPU, and imports prefill states from the framework cache; attention runs through FlashAttention’s KV-cache decode API or the generated paged decode-attention kernel. One exception is the KV append: FlashAttention’s in-kernel bfloat16 (bf16) append is $\sim 4\times$ slower than fp16 (0.73 vs. 0.18 ms per decode step), so a small scatter `kv_append` kernel writes the new K/V vectors and FlashAttention takes its no-append fast path with the sequence length advanced by one, which is semantically identical to the fused call, and bf16 generation becomes slightly faster than fp16 as a result.

For the multi-request adaptation, we implement serving-grade mechanisms end to end. For memory, a paged KV cache allocates at block granularity, and requests are pre-empted by recomputation or by swapping KV blocks to host memory under an aggressive admission policy. For scheduling, the engine batches requests continuously and mixes

Model	Framework	Workload (input/output)					Avg. Speedup
		i128/o128	i2048/o512	i8192/o4096	i12288/o8192	i20480/o16384	
Qwen3-4B	PyTorch	4765	18974	151807	303239	603417	–
	<code>torch.compile</code>	1707/2.8×	7092/2.7×	76237/2.0×	202430/1.5×	602062/1.0×	1.99×
	Ours	824/5.8×	3482/5.4×	30925/4.9×	66095/4.6×	150819/4.0×	4.95×
Falcon3-7B	PyTorch	2880	11694	91589	186789	–	–
	<code>torch.compile</code>	1457/2.0×	6809/1.7×	81197/1.1×	201527/0.9×	–	1.44×
	Ours	1194/2.4×	5089/2.3×	43022/2.1×	90409/2.1×	–	2.23×
Mistral-7B	PyTorch	3164	12822	100928	203672	–	–
	<code>torch.compile</code>	1510/2.1×	7401/1.7×	94048/1.1×	240877/0.8×	–	1.44×
	Ours	1212/2.6×	5086/2.5×	43130/2.3×	89895/2.3×	–	2.43×
GLM-4-9B	PyTorch	5023	19966	161703	320419	640323	–
	<code>torch.compile</code>	2105/2.4×	8619/2.3×	95306/1.7×	233744/1.4×	641812/1.0×	1.75×
	Ours	1668/3.0×	6856/2.9×	57668/2.8×	115555/2.8×	239863/2.7×	2.83×
Qwen3-VL-2B	PyTorch	3575	14181	113733	229348	456021	–
	<code>torch.compile</code>	1244/2.9×	5093/2.8×	41139/2.8×	103825/2.2×	303122/1.5×	2.43×
	Ours	416/8.6×	1772/8.0×	16626/6.8×	36562/6.3×	87523/5.2×	6.98×

Table 1: Representative single-request end-to-end generation latency (ms) and speedup ratio measured against PyTorch eager execution. Workload denotes input/output length; ‘–’ marks workloads exceeding the model’s maximum context length. The last column reports the average speedup over the listed workloads.

Model	Framework	Workload (input/output)					Avg. Speedup
		i128/o128	i2048/o512	i8192/o4096	i12288/o8192	i20480/o16384	
Qwen3-4B	vLLM	905	3727	32690	69769	158059	–
	Ours	824/1.10×	3482/1.07×	30925/1.06×	66095/1.06×	150819/1.05×	1.07×
Llama-3.2-3B	vLLM	726	2994	26125	55615	125507	–
	Ours	652/1.11×	2749/1.09×	24321/1.07×	51968/1.07×	118497/1.06×	1.08×
Falcon3-7B	vLLM	1329	5496	46082	96415	–	–
	Ours	1194/1.11×	5089/1.08×	43022/1.07×	90409/1.07×	–	1.08×
Mistral-7B	vLLM	1346	5557	46788	97519	–	–
	Ours	1212/1.11×	5086/1.09×	43130/1.08×	89895/1.08×	–	1.09×
Qwen3-VL-2B	vLLM	508	2103	18997	41264	96700	–
	Ours	416/1.22×	1772/1.19×	16626/1.14×	36562/1.13×	87523/1.10×	1.16×

Table 2: Representative single-request end-to-end generation latency (ms) and speedup ratio measured against vLLM.

Model	Workload (input/output)					Avg. Speedup
	i2k/o1k n32m32	i2k/o4k n64m64	i4k/o1k n32m32	i8k/o8k n40m40	i10k/o10k n30m30	
Qwen3-4B	1.02×	1.03×	1.04×	1.06×	1.06×	1.04×
Llama-3.2-3B	1.03×	1.04×	1.05×	1.04×	1.27×	1.09×
Mistral-7B	1.05×	–	1.05×	1.02×	–	1.04×
Qwen3-VL-2B	1.04×	1.06×	1.06×	1.09×	1.34×	1.12×

Table 3: Multi-request end-to-end throughput speedup over vLLM on the uniform suite. All requests in a tier share the same input/output lengths; tiers are labeled $i\langle in \rangle/o\langle out \rangle n\langle n \rangle m\langle m \rangle$, where $k = 1024$ tokens, n is the total number of requests, and m is the maximum number of concurrently admitted requests. ‘–’ marks tiers exceeding the model’s KV-memory budget.

chunked prefill with decode steps. For execution, decode steps replay as *bucketed full-step CUDA Graphs*, where one graph per batch-size bucket covers the entire batched step, eliminating per-token launch and dispatch overhead; memory budgeting is parameterized per model.

Experiments

We evaluate AGENTCOMPILE on single-request end-to-end generation and multi-request serving, quantifying the acceleration over PyTorch eager, `torch.compile`, and vLLM; kernel-level quality, compiler-pipeline statistics, and

Model	Workload (KV pool $\times$ pressure)						Avg. Speedup
	12M	16M	60H	30HH	40HH	60HH	
Qwen3-4B	1.02 $\times$	1.01 $\times$	1.06 $\times$	1.08 $\times$	1.12 $\times$	1.12 $\times$	1.07$\times$
Llama-3.2-3B	1.05 $\times$	1.02 $\times$	1.03 $\times$	1.13 $\times$	1.12 $\times$	1.09 $\times$	1.07$\times$
Falcon3-7B	1.03 $\times$	1.03 $\times$	1.02 $\times$	1.08 $\times$	1.01 $\times$	1.04 $\times$	1.04$\times$
Mistral-7B	1.02 $\times$	1.04 $\times$	1.01 $\times$	1.09 $\times$	1.13 $\times$	1.05 $\times$	1.06$\times$
Qwen3-VL-2B	1.09 $\times$	1.03 $\times$	1.01 $\times$	1.18 $\times$	1.16 $\times$	1.03 $\times$	1.08$\times$

Table 4: Multi-request end-to-end throughput speedup over vLLM on the ragged swap suite. Each tier issues a mixed-length request pool against a paged-KV pool, whose size is given by the number in the tier label (in GB, e.g., 12M runs against a 12GB pool). M (medium-pressure) tiers use requests that declare large output budgets but terminate early, so the declared load oversubscribes the KV pool by $\sim 3\times$ and forces admission and preemption decisions; H (long) tiers use genuinely long mixed requests ($3\times$ length scale, prompts up to $\sim 12k$ tokens); HH (extreme) tiers scale request lengths $5\text{--}6\times$. Both systems serve the identical request pool. Details in appendix.

Framework Change	Workload (i/o)					Avg. Speedup
	i128/o128	i2048/o512	i8192/o4096	i12288/o8192	i20480/o16384	
baseline	4765	18974	151807	303239	603417	—
+ CL & GEMV & StaticCache	2986	11908	95645	192952	386487	1.58 $\times$
+ FAKV & QKV_proj & Graph	1160	4826	41245	86754	191991	2.32 $\times$
+ LLM & LLM-generated kernels	824	3482	30925	66095	150819	1.34 $\times$
= full system	—	—	—	—	—	4.95 $\times$

Table 5: Qwen3-4B stepwise runtime ablation for single-request autoregressive generation (batch size 1). Columns denote input/output sequence lengths, and values report mean latency in milliseconds. “baseline” denotes PyTorch eager with FlashAttention; “CL” denotes CUDALinear, “GEMV” denotes template-generated single-op GEMV kernels (compiler tier, not LLM-authored); “FAKV” denotes FlashAttention with KV-cache update and our KV-cache manager (including a custom bf16 kv_append kernel replacing FlashAttention’s slow bf16 append), “QKV_proj” denotes fused QKV projection, “Graph” denotes CUDA Graph replay; “LLM” denotes the LLM acting as a semantic advisor to the compiler pipeline (labels and candidate hints, no kernel authoring); “LLM-generated kernels” denotes the five families of LLM-authored kernels.

encoder-only vision results are reported in the appendix. All LLM stages use Claude-Opus-4.8.

Setup. Experiments use an NVIDIA A800 SXM4 80GB GPU with an Intel Xeon Platinum 8358 host. We evaluate six model families: Qwen3-4B, Llama-3.2-3B-Instruct, Falcon3-7B-Instruct, Mistral-7B-Instruct-v0.3, GLM-4-9B-chat, and Qwen3-VL-2B-Instruct (text stack), in bfloat16. The software stack, latency accounting, timing protocol, and the execution path of each system are detailed in the appendix.

Single-Request End-to-End Latency Comparisons

Figure 4 and Tables 1–2 compare PyTorch eager, torch.compile+FlashAttention, vLLM, and AGENTCOMPILE across representative workloads: AGENTCOMPILE is 2.23–6.98 $\times$ faster than PyTorch eager on average, consistently above torch.compile (1.44–2.43 $\times$), whose advantage collapses on long workloads, and faster than vLLM on every measured workload (1.07–1.16 $\times$ on average).

Figure 4 shows that AGENTCOMPILE’s speedup over PyTorch eager generally becomes more pronounced as output length increases, especially on long-input workloads. This reflects the decode-oriented design of the runtime: paged KV management, full-step CUDA Graph replay, and generated GEMV/fused kernels reduce per-token overhead across

long autoregressive loops.

In contrast, PyTorch eager retains substantial framework dispatch and dynamic-cache overhead. torch.compile is effective on short-input workloads, where fusion and compiled graph fragments reduce framework and kernel-launch overhead; as input length increases, computation and KV cache traffic dominate, and graph breaks, dynamic cache updates, and shape changes further reduce graph reuse. At very long outputs, the speedup gradually decreases because the growing KV cache makes attention increasingly dominant and less differentiated across systems. Nevertheless, our AGENTCOMPILE still substantially outperforms both PyTorch eager and torch.compile by a wide margin in this regime.

The vLLM comparison in Table 2 should be interpreted carefully, because vLLM and AGENTCOMPILE optimize different and independent parts of the inference stack. vLLM is a serving-oriented runtime whose strengths lie in paged KV management, request scheduling, and production decode execution, whereas AGENTCOMPILE’s contribution is the kernel side: compiler-selected and LLM-generated kernels, fused decode components, and full-step CUDA Graph replay. The two systems thus improve largely complementary parts of the inference stack. In principle, the kernels studied here could also serve inside a scheduler such as vLLM’s.

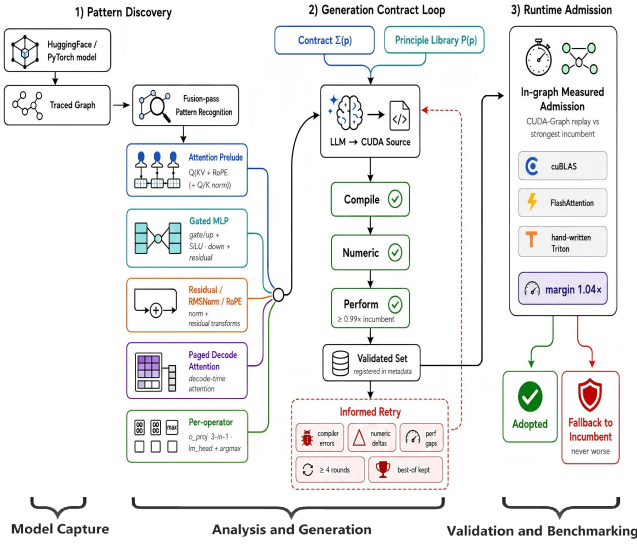


Figure 3: Generation pipeline of the five LLM-generated kernel families: recognized patterns branch into per-family contracts $\Sigma(p)$ and source-free principle libraries $P(p)$; the LLM produces complete CUDA source that must pass the compile, numeric, and performance gates, with informed retry on structured failure feedback; validated kernels reach production only through in-graph measured admission, and losers fall back. Failure modes flow back into $P(p)$ as falsification clauses; the decode-attention kernel propagates across models as a re-validated seed.

Multi-Request Throughput vs. vLLM

We further compare the continuous-batching engine against vLLM under a *uniform* suite and a *ragged swap* suite (Tables 3 and 4; workload construction in the captions). The engine is faster than vLLM on every runnable tier of both suites; the advantage widens on the long-output uniform tiers, where bucketed full-step graph replay amortizes per-step host overhead over many decode steps, and on the extreme long-context (HH) swap tiers, where aggressive admission with KV-swap preemption keeps the GPU saturated.

Ablation Studies

Table 5 reports a stepwise Qwen3-4B ablation. `CUDALinear` with single-op GEMV kernels and a static KV cache (replacing the PyTorch `DynamicCache`) yields $1.58\times$ by removing framework and dynamic-cache overhead. Adding FAKV, fused QKV projection, and CUDA Graph replay is the largest step ($2.32\times$): FAKV is near-neutral in eager mode but provides the graph-capturable decode path that full-step replay requires; capturing a CUDA Graph on the SDPA (scaled dot-product attention) + `StaticCache` path instead plateaus at 1318 ms on i128/o128, versus 824 ms with FAKV (both measured in the full-system configuration). The LLM tier contributes a further $1.34\times$, for $4.95\times$ overall. Stage averages are per-workload means, so their product is not exactly equal to the full-system average, which is computed directly from the baseline and

full-system latencies.

Conclusion

This paper presented AGENTCOMPILE, a compiler framework in which an LLM directly authors decode-critical CUDA kernels under compiler-defined contracts. The key idea is to separate authorship from admission: the LLM writes complete kernels from distilled optimization principles, while compilation, interface, numerical gates, and runtime in-graph measured selection with fallback decide what actually runs. Combined with a serving-grade runtime, the system outperforms PyTorch eager and `torch.compile` on single-request generation, and vLLM on both single-request generation and multi-request serving. The methodology of hand-writing once, distilling to principles, and generating everywhere offers a practical path for accumulating kernel-engineering knowledge in a form that LLMs can reuse.

Limitations

AGENTCOMPILE is a prototype. It targets transformer-style inference regions, and unsupported regions fall back to the original framework or library path. To further improve the performance of the proposed method, we will explore more refined and efficient CUDA kernels in future work.

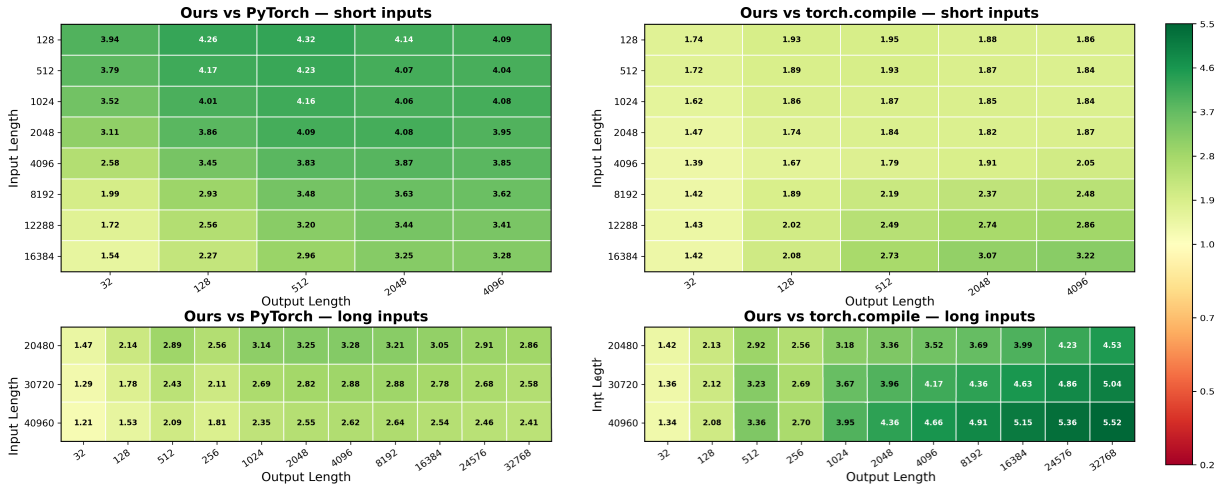


Figure 4: Input/output-length scaling comparison on Llama-3.2-3B-Instruct across PyTorch eager, torch.compile+FlashAttention, and AGENTCOMPILE.

References

- Ansel, J.; Yang, E.; He, H.; Gimselshein, N.; Jain, A.; Voznesensky, M.; Bao, B.; Bell, P.; Berard, D.; Burovski, E.; Chauhan, G.; Chourdia, A.; Constable, W.; Desmaison, A.; DeVito, Z.; Ellison, E.; Feng, W.; Gong, J.; Gschwind, M.; Hirsh, B.; Huang, S.; Kalambarkar, K.; Kirsch, L.; Lazos, M.; Lezcano, M.; Liang, Y.; Liang, J.; Lu, Y.; Luk, C.; Maher, B.; Pan, Y.; Puhrsch, C.; Reso, M.; Saroufim, M.; Siraichi, M. Y.; Suk, H.; Suo, M.; Tillet, P.; Wang, E.; Wang, X.; Wen, W.; Zhang, S.; Zhao, X.; Zhou, K.; Zou, R.; Mathews, A.; Chanan, G.; Wu, P.; and Chintala, S. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Byte-code Transformation and Graph Compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 929–947. Association for Computing Machinery.
- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. d. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; Ray, A.; Puri, R.; Krueger, G.; Petrov, M.; Khlaaf, H.; Sastry, G.; Mishkin, P.; Chan, B.; Gray, S.; Ryder, N.; Pavlov, M.; Power, A.; Kaiser, L.; Bavarian, M.; Winter, C.; Tillet, P.; Petroski Such, F.; Cummings, D.; Plappert, M.; Chantzis, F.; Barnes, E.; Herbert-Voss, A.; Guss, W. H.; Nichol, A.; Paino, A.; Tezak, N.; Tang, J.; Babuschkin, I.; Balaji, S.; Jain, S.; Saunders, W.; Hesse, C.; Carr, A. N.; Leike, J.; Achiam, J.; Misra, V.; Morikawa, E.; Radford, A.; Knight, M.; Brundage, M.; Murati, M.; Mayer, K.; Welinder, P.; McGrew, B.; Amodei, D.; McCandlish, S.; Sutskever, I.; and Zaremba, W. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374.
- Chen, T.; Moreau, T.; Jiang, Z.; Zheng, L.; Yan, E.; Shen, H.; Cowan, M.; Wang, L.; Hu, Y.; Ceze, L.; Guestrin, C.; and Krishnamurthy, A. 2018a. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 578–594. USENIX Association.
- Chen, T.; Zheng, L.; Yan, E.; Jiang, Z.; Moreau, T.; Ceze, L.; Guestrin, C.; and Krishnamurthy, A. 2018b. Learning to Optimize Tensor Programs. In Bengio, S.; Wallach, H.; Larochelle, H.; Grauman, K.; Cesa-Bianchi, N.; and Garnett, R., eds., *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc.
- Cummins, C.; Seeker, V.; Grubisic, D.; Elhoushi, M.; Liang, Y.; Roziere, B.; Gehring, J.; Gloeckle, F.; Hazelwood, K.; Synnaeve, G.; and Leather, H. 2023. Large Language Models for Compiler Optimization. arXiv:2309.07062.
- Dao, T.; Fu, D.; Ermon, S.; Rudra, A.; and Ré, C. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In Koyejo, S.; Mohamed, S.; Agarwal, A.; Belgrave, D.; Cho, K.; and Oh, A., eds., *Advances in Neural Information Processing Systems*, volume 35, 16344–16359. Curran Associates, Inc.
- Jia, Z.; Padon, O.; Thomas, J.; Warszawski, T.; Zaharia, M.; and Aiken, A. 2019. TASO: Optimizing Deep Learning Computation with Automatic Generation of Graph Substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 47–62. Association for Computing Machinery.
- Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C. H.; Gonzalez, J. E.; Zhang, H.; and Stoica, I. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–626. Association for Computing Machinery.
- Lattner, C.; Amini, M.; Bondhugula, U.; Cohen, A.; Davis, A.; Pienaar, J.; Riddle, R.; Shpeisman, T.; Vasilache, N.; and Zinenko, O. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2–14. IEEE.

- Li, R.; Ben Allal, L.; Zi, Y.; Muennighoff, N.; Kocetkov, D.; Mou, C.; Marone, M.; Akiki, C.; Li, J.; Chim, J.; Liu, Q.; Zheltonozhskii, E.; Zhuo, T. Y.; Wang, T.; Dehaene, O.; Davaadorj, M.; Lamy-Poirier, J.; Monteiro, J.; Shliazhko, O.; Gontier, N.; Meade, N.; Zebaze, A.; Yee, M.-H.; Umapathi, L. K.; Zhu, J.; Lipkin, B.; Oblokulov, M.; Wang, Z.; Murthy, R.; Stillerman, J.; Patel, S. S.; Abulkhanov, D.; Zocca, M.; Dey, M.; Zhang, Z.; Fahmy, N.; Bhattacharyya, U.; Yu, W.; Singh, S.; Luccioni, S.; Villegas, P.; Kunakov, M.; Zhdanov, F.; Romero, M.; Lee, T.; Timor, N.; Ding, J.; Schlesinger, C.; Schoelkopf, H.; Ebert, J.; Dao, T.; Mishra, M.; Gu, A.; Robinson, J.; Anderson, C. J.; Dolan-Gavitt, B.; Contractor, D.; Reddy, S.; Fried, D.; Bahdanau, D.; Jernite, Y.; Muñoz Ferrandis, C.; Hughes, S.; Wolf, T.; Guha, A.; von Werra, L.; and de Vries, H. 2023. StarCoder: May the Source Be with You! *Transactions on Machine Learning Research*.
- Li, Y.; Choi, D.; Chung, J.; Kushman, N.; Schrittwieser, J.; Leblond, R.; Eccles, T.; Keeling, J.; Gimeno, F.; Lago, A. D.; Hubert, T.; Choy, P.; de Masson d’Autume, C.; Babuschkin, I.; Chen, X.; Huang, P.-S.; Welbl, J.; Gowal, S.; Cherepanov, A.; Molloy, J.; Mankowitz, D. J.; Sutherland Robson, E.; Kohli, P.; de Freitas, N.; Kavukcuoglu, K.; and Vinyals, O. 2022. Competition-Level Code Generation with AlphaCode. *Science*, 378(6624): 1092–1097.
- Ma, L.; Xie, Z.; Yang, Z.; Xue, J.; Miao, Y.; Cui, W.; Hu, W.; Yang, F.; Zhang, L.; and Zhou, L. 2020. Rammer: Enabling Holistic Deep Learning Compiler Optimizations with rTasks. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 881–897. USENIX Association.
- Ni, A.; Allamanis, M.; Cohan, A.; Deng, Y.; Shi, K.; Sutton, C.; and Yin, P. 2024. NExT: Teaching Large Language Models to Reason about Code Execution. In *International Conference on Learning Representations*.
- Niu, W.; Guan, J.; Wang, Y.; Agrawal, G.; and Ren, B. 2021. DNNFusion: Accelerating Deep Neural Networks Execution with Advanced Operator Fusion. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 883–898. Association for Computing Machinery.
- NVIDIA. 2024. CUDA C++ Programming Guide: CUDA Graphs. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/cuda-graphs.html>. Accessed: 2026-04-30.
- OpenXLA Project. 2024. XLA: Optimizing Compiler for Machine Learning. <https://openxla.org/xla>. Accessed: 2026-04-30.
- Ouyang, A.; Guo, S.; Arora, S.; Zhang, A. L.; Hu, W.; Ré, C.; and Mirhoseini, A. 2025. KernelBench: Can LLMs Write Efficient GPU Kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 47356–47415. PMLR.
- Reed, J.; DeVito, Z.; He, H.; Ussery, A.; and Ansel, J. 2022. torch.fx: Practical Program Capture and Transformation for Deep Learning in Python. In Marculescu, D.; Chi, Y.; and Wu, C., eds., *Proceedings of Machine Learning and Systems*, volume 4, 638–651.
- Shi, Y.; Yang, Z.; Xue, J.; Ma, L.; Xia, Y.; Miao, Z.; Guo, Y.; Yang, F.; and Zhou, L. 2023. Welder: Scheduling Deep Learning Memory Access via Tile-graph. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 701–718. USENIX Association.
- Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; and Levine, S., eds., *Advances in Neural Information Processing Systems*, volume 36, 8634–8652. Curran Associates, Inc.
- Tillet, P.; Kung, H.-T.; and Cox, D. D. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 10–19. Association for Computing Machinery.
- Yu, G.-I.; Jeong, J. S.; Kim, G.-W.; Kim, S.; and Chun, B.-G. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 521–538. USENIX Association.
- Zheng, L.; Jia, C.; Sun, M.; Wu, Z.; Yu, C. H.; Haj-Ali, A.; Wang, Y.; Yang, J.; Zhuo, D.; Sen, K.; Gonzalez, J. E.; and Stoica, I. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 863–879. USENIX Association.
- Zheng, Z.; Yang, X.; Zhao, P.; Long, G.; Zhu, K.; Zhu, F.; Zhao, W.; Liu, X.; Yang, J.; Zhai, J.; Song, S. L.; and Lin, W. 2022. AStitch: Enabling a New Multi-Dimensional Optimization Space for Memory-Intensive ML Training and Inference on Modern SIMT Architectures. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 359–373. Association for Computing Machinery.
- Zhu, H.; Wu, R.; Diao, Y.; Ke, S.; Li, H.; Zhang, C.; Xue, J.; Ma, L.; Xia, Y.; Cui, W.; Yang, F.; Yang, M.; Zhou, L.; Cidon, A.; and Pekhimenko, G. 2022. ROLLER: Fast and Efficient Tensor Compilation for Deep Learning. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 233–248. USENIX Association.

Supplementary Implementation Notes

This appendix records implementation details that support, but do not repeat, the main method. The frontend supports single-file modules, project directories, and HuggingFace identifiers, and resolves model classes through the framework’s authoritative configuration registry; command-line contracts require the model path, class, and identifier to be mutually consistent. The IR builder normalizes operator names and records numerical contracts such as precise softmax/norm behavior, associative reduction or matmul behavior, and relaxed elementwise behavior. The analyzer calls the LLM only for regions whose rule-based pattern is mixed, complex GEMM-epilogue, or reduction-pointwise

and whose region size justifies semantic analysis. Accepted LLM metadata is stored as region hints and planner preferences; rejected advice is ignored without changing the graph. Generated fusion kernels are registered in emitted metadata under per-model name prefixes; standalone generator entry points can regenerate a single kernel class and merge its registration without re-running the full pipeline. The hand-written paged decode-attention reference distilled in the main text is a warp-autonomous Tensor-Core pipeline using `m16n8k16mma` fragments whose accumulator layout is lane-isomorphic to the operand layout, enabling in-fragment softmax, with `ldmatrix` loads, padded shared-memory rows to avoid bank conflicts, and a fused fence-based merge that removes a separate reduction kernel.

The planner constructs candidates using fusion, schedule, memory, and parallel sub-planners. Each candidate specifies an implementation family and realizable parameters: fusion boundary, template type, tile size, thread count, vector width, reduction mapping, shared-memory policy, and launcher interface. Candidate generation is deliberately bounded: every plan must have a known template, a compatible launcher signature, and hardware-valid parameters before code generation. The code generator includes templates for elementwise chains, reductions, softmax, normalization, GEMM, Tensor-Core GEMM, GEMV, and selected fused epilogues. Validation combines compilation, interface checks, empirical numerical checks when available, and optional memory-safety tooling. The benchmark selector records latency for validated candidates and the emitter writes source, headers, wrappers, metadata, and build scripts.

Kernel-Level Quality of LLM-Generated Kernels

Table 7 reports the best validated kernel per class, measured in-graph against the existing implementation it replaces. Two regimes emerge. Where fusion replaces a multi-op framework path, generated kernels win by large margins ($3.7\text{--}4.3\times$ for RoPE). Where the existing implementation is already near the bandwidth roofline, they cluster at parity to modest gains ($0.96\text{--}1.12\times$); the paged decode-attention kernel, which is generated from distilled principles alone and re-validated per model from a cross-model seed, reaches $0.955\text{--}0.975\times$ of FlashAttention on four families (GLM-4’s 32:2 grouped-query configuration reaches $0.45\times$ and is rejected). Kernels that fail any gate or lose the measured comparison simply retain their baselines.

Encoder-Only Vision Results

The pure-vision encoder DINOv3-ViT-H+/16 has no autoregressive decode, no KV cache, and no vLLM support, so we compare the assembled encoder, built from the same generation machinery through the encoder kernel family, against PyTorch eager across batch sizes (Table 8). Gains are largest at small batches, where framework dispatch and small-kernel launch overhead dominate, and settle near parity as the workload becomes GEMM-bound; assembled features match the eager reference with cosine similarity 0.9998.

Compiler-Pipeline Statistics

Analysis of Intermediate States Table 6 reports the compiler runs for all seven families. Each run traces the model into an internal IR, partitions it into regions, generates candidate CUDA implementations, validates them, benchmarks validated candidates, and emits deployment artifacts; executable kernels are thus selected through materialization, compilation, validation, benchmarking, and emission rather than textual plausibility. Every generated candidate passes validation (e.g., the Qwen3-4B log records maximum absolute and relative differences of $0.00e+00$ across its 174 validations), one implementation is selected per region (selected-kernel latencies of $7.78\text{--}13.97\ \mu\text{s}$ on Llama), and the DINOv3 assembled encoder matches the eager reference with feature cosine similarity 0.9998.

Per-Model Adaptations

Table 9 summarizes the model-specific adaptations required to extend AGENTCOMPILE from its original Qwen3 and Llama models to seven model families. Most adaptations are one-time frontend or generator generalizations that benefit all subsequent models; structural gates ensure that unsupported patterns are skipped safely rather than mis-generated.

Systems Compared

Table 10 summarizes the runtime path of each system. PyTorch eager uses HuggingFace/PyTorch generation with DynamicCache, framework dispatch, cuBLAS linear layers, and PyTorch SDPA when applicable. `torch.compile` with FlashAttention evaluates a PyTorch-native optimization path with Inductor fragments and a FlashAttention patch, but it still follows the framework generation path with dynamic KV cache growth. vLLM represents a serving runtime with PagedAttention, paged KV blocks, scheduling, and CUDA Graph replay for common decode shapes.

AGENTCOMPILE uses its own serving runtime: one-time preparation allocates the paged KV pool from a per-model memory budget, loads generated and baseline kernels under the model’s name prefix, performs in-graph measured admission per kernel site, and captures bucketed full-step decode graphs. Timed prefill uses cuBLAS and FlashAttention (chunked when needed); timed decode replays the captured graph, executing admitted LLM-generated kernels with hand-written or library fallbacks at non-admitted sites; a use audit records the implementation serving each step.

Measurement protocol. The software stack includes HuggingFace Transformers, FlashAttention-2, Triton, vLLM, and CUDA compilation for generated kernels. Reported latency includes timed prefill and decode and excludes one-time preparation such as cache allocation, attention patching, extension loading, warmup, and CUDA Graph capture. Single-request timing uses two warmup iterations and ten measured repetitions. Workloads exceeding a model’s maximum context length are excluded for all systems, since positional extrapolation makes such outputs meaningless.

Table 6: Compiler-pipeline statistics for the seven model families. Validated denotes candidates that passed compilation and numerical checks under the logged validation protocol; one implementation is selected per region.

Run	IR nodes	Regions	Plans / candidates	Validated	Selected	Emitted artifacts
Qwen3-4B	236	34	174	174/174	34	7 files / 43 signatures
Llama-3.2-3B-Instruct	192	34	174	174/174	34	7 files / 43 signatures
Falcon3-7B-Instruct	192	34	227	227/227	34	7 files / 43 signatures
Mistral-7B-Instruct-v0.3	201	37	249	249/249	37	7 files / 46 signatures
GLM-4-9B-chat	238	50	347	347/347	50	7 files / 57 signatures
Qwen3-VL-2B-Instruct	291	34	227	227/227	34	7 files / 43 signatures
DINOv3-ViT-H+/16	170	49	384	384/384	49	7 files / 51 signatures

Table 7: Best validated LLM-generated kernel per class, relative to the existing implementation it replaces, under the in-graph measurement protocol (>1 means the generated kernel is faster). The replaced implementations fall into three groups: (a) *expert kernels*: attention preludes are measured against our hand-written fused reference kernels, and paged decode attention against FlashAttention; (b) *library paths*, i.e., what production runs without the LLM tier: MLP rows against cuBLAS, and `lm_head+argmax` against the cuBLAS + `torch.argmax` chain; (c) *framework operator paths*: the RoPE row is measured against the multi-op framework path it replaces, a weaker baseline than group (a)/(b), which explains the large ratio. All MLP and `lm_head` rows are the single-token ($M=1$) decode variants.

Kernel class	Best model	Ratio
Attention prelude (QKV+RoPE)	Falcon3-7B	$1.03\times$
Attention prelude (+Q/K norm)	Qwen3-VL-2B	$1.02\times$
Paged decode attention	Qwen3-VL-2B	$0.975\times$
<code>lm_head+argmax</code> ($M=1$)	Llama-3.2-3B	$1.07\times$
MLP gate-up + SiLU ($M=1$)	Qwen3-4B	$1.12\times$
MLP down + residual ($M=1$)	GLM-4-9B	$1.01\times$
Standalone RoPE	Qwen3-4B	$4.27\times$

Multi-Request Workload Construction

Each tier of the ragged swap suite (Table 4 in the main paper) is defined by a paged-KV pool size and a workload mix: the number in the tier label is the pool size in GB (e.g., 12M runs against a 12GB pool), and the letter denotes the mix. The base mix is a pool of mixed-length requests whose prompts range from a few hundred to about 4k tokens and whose outputs reach about 2k tokens. H (long) tiers scale all lengths by $3\times$, with prompts up to $\sim 12k$ tokens, and HH (extreme) tiers scale them by $5\text{--}6\times$; in both, the declared and actual lengths are equal. M (medium-pressure) tiers instead model early-stopping traffic: each request declares a large output budget but terminates early at its true generation length, and the pool is filled until the declared footprint (prompt plus declared maximum output) reaches about $3\times$ the KV-pool capacity, so the scheduler must over-commit, preempt, and swap based on declared rather than actual lengths. The maximum number of concurrently admitted requests is derived from the remaining GPU memory after subtracting the model weights, the KV pool, a safety margin, and a per-request activation estimate. Both systems serve the same request pool, that is, the identical

prompts, length pairs, and arrival order, and we report end-to-end throughput over the whole pool. The uniform suite (Table 3 in the main paper) is the simpler counterpart: all requests in a tier share one input/output length, and the tier fixes the total number of requests and the maximum concurrency.

Extended Related Work

Neural-network compilers: TVM, XLA, PyTorch 2, MLIR, and Triton expose graph- or tensor-level abstractions for lowering models to accelerators (Chen et al. 2018a; OpenXLA Project 2024; Ansel et al. 2024; Lattner et al. 2021; Tillet, Kung, and Cox 2019); AutoTVM, Ansor, ROLLER, Welder, TASO, DNNFusion, AStitch, and Rammer automate schedules, rewrites, memory planning, or graph execution (Chen et al. 2018b; Zheng et al. 2020; Zhu et al. 2022; Shi et al. 2023; Jia et al. 2019; Niu et al. 2021; Zheng et al. 2022; Ma et al. 2020). AGENTCOMPILE is complementary, adding an LLM-authored kernel tier on top of compiler-defined constraints, validation, and measurement.

Inference runtimes and kernels: On the serving side, vLLM and ORCA improve efficiency through scheduling, batching, and memory management (Kwon et al. 2023; Yu et al. 2022), with CUDA Graphs and FlashAttention providing launch-overhead and attention-kernel optimizations (NVIDIA 2024; Dao et al. 2022); AGENTCOMPILE implements the same serving-grade mechanisms but sources its decode kernels from contract-gated LLM generation rather than a fixed expert-written library.

LLMs for code and compiler optimization: Among applications of LLMs to code and compiler optimization (Chen et al. 2021; Li et al. 2022, 2023; Shinn et al. 2023; Ni et al. 2024; Cummins et al. 2023), KernelBench evaluates direct LLM generation of GPU kernels (Ouyang et al. 2025); AGENTCOMPILE differs in prompting from distilled principle libraries (including falsified dead ends) rather than task descriptions or reference source, and in requiring every kernel to pass compiler contracts, acceptance thresholds, and runtime measured admission with a never-worse fallback.

Table 8: End-to-end inference speedup of the assembled DINOv3 encoder over PyTorch eager. B denotes the batch size; the last column reports the average over the eight batch sizes.

Model	Batch size B								Avg. Speedup
	1	2	4	8	16	32	64	96	
dinov3-vith16plus -pretrain-lvd1689m	2.57×	1.93×	1.11×	1.06×	1.03×	1.03×	1.02×	1.02×	1.35×

Table 9: Model-specific adaptations and their measured effects. Ratios are generated-kernel speed vs. the baseline implementation unless noted.

Model	Adaptations	Measured effect
Mistral-7B-Instruct-v0.3	Sliding-window attention wired through the shared Llama-family container	Faster than vLLM on all measured workloads (1.08–1.11×
Falcon3-7B-Instruct	Architecture resolved from configuration metadata (fixing a silent wrong-class capture); Llama-family container reuse; per-model activation-budget calibration	Attention prelude 1.03×; GEMV 1.04×; zero OOM-driven concurrency throttling
GLM-4-9B-chat	Merged <code>gate_up_proj</code> layout adaptation (row-sliced zero-copy weight views feeding the fused MLP kernels); token-id clamping under configuration shrinking; dual defense for partial rotary (runtime width check + generator gate)	53/53 kernels validated; MLP down+residual 1.01×; end-to-end outputs match reference
Qwen3-VL-2B-Instruct	Text-only tracing for conditional-generation classes; recursive sub-configuration shrinking; text stack reuses the Qwen3 container	Q/K-norm RoPE prelude 1.02×; GEMV 1.01×
DINOv3-ViT-H+/16	Pure-vision encoder assembly layer: per-region numerical gating (cosine ≥ 0.9995), bias folding, CUDA Graph replay with per-operator fallback	51/51 kernels validated; assembled features cosine 0.9998 vs. eager

Table 10: Runtime execution paths of compared systems during autoregressive generation (FA = FlashAttention).

System	Linear / MLP path	Attention path	KV cache	Decode control
PyTorch Eager (FA)	cuBLAS	SDPA (flash backend)	DynamicCache	Python loop (eager)
<code>torch.compile</code> +FA	Inductor / Triton	FlashAttention	DynamicCache	Inductor graph
vLLM	cuBLAS + fused QKV	PagedAttention	Paged blocks	CUDA Graph replay
Ours (Single-Request)	CUDALinear + generated GEMV + LLM-generated fused kernels	LLM-generated paged decode attention (FA fallback)	FlashKVCacheManager	CUDA Graph replay
Ours (Multi-Request)	Same as single-request	Same as single-request	Paged blocks + swap preemption	Bucketed full-step CUDA Graph replay